Master's Thesis

Applications of Reinforcement Learning to Stateful Emulator-Based Greybox Fuzzing

Bastian Engel

July 20, 2026

Abstract

Contents

Chapter 1

Introduction

1.1 Fuzzing

1.1.1 Overview and Motivation

Fuzz testing, or *fuzzing* for short, is a widely deployed method for automatic software testing and vulnerability discovery that makes use of a diverse set of tools [26]. Its goal is to find inputs to a Program Under Test (PUT) that trigger a certain behavior, often a crash that might then lead to the discovery of bugs or security vulnerabilities.

The technique was first introduced in the early 1990s as a way of testing the reliability of UNIX utilities [27]. The authors were inspired by spurious characters being introduced to a dial-up line causing crashes of common programs on a remote workstation. This lad to the creation of a simple command-line utility *fuzz* that produced random input strings that were then sent to various utilities in the hope of causing a crash in the form of a core dump or a hang. They managed to find inputs that caused between 24% and 33% of the tested utility programs to crash or hang. By analyzing these inputs, bugs could be identified and fixed, thereby contributing to the reliability of the system.

The same approach is still in use today to find security vulnerabilities in input-processing code, especially memory-safety bugs in C/C++. For example, Google's OSS-Fuzz "aims to make common open source software more secure and stable by combining modern fuzzing techniques with scalable, distributed execution" [2]. It supports a variety of target languages and is therefore widely applicable. OSS-Fuzz has helped to fix over 13,000 security vulnerabilities and 50,000 bugs across 1,000 projects, proving the importance of modern fuzzing techniques for ensuring reliability and security.

1.1.2 Core Terminology

We will introduce some general definitions that mainly follow Manes et al. [26].

1. *Program Under Test (PUT):* The program under test is an executable software artifact like a command-line utility or library together with an interface that consumes input. In our case it denotes a binary executed through the MOGI emulatorChapter 5.

2. *Security Policy:* A security policy defines which observable behaviors of the PUT are acceptable. Each execution either satisfies or violates a given security policy. A common security policy deems all executions that result in crashes like segmentation fauls to be violations.

3. *Bug Oracle:* A bug oracle is a program that decides whether a given input to the PUT violates a given security policy. A widely used bug oracle might deem an execution a violation if the PUT crashes or hangs.

4. *Fuzzing:* Fuzzing denotes the process of feeding the PUT with inputs sampled from an input space. This input space may include expected and valid inputs as well as invalid, unexpected inputs in the hope of violating a given security policy.

5. *Fuzz Testing:* Fuzz testing denotes the software testing technique that makes use of fuzzing.

6. *Fuzzer:* A fuzzer is a program that performs fuzz testing on a PUT.

7. *Fuzz Campaign:* A fuzz campaign is a specific execution of a fuzzer on a PUT with a specific security policy. Its goal is to find bugs in the PUT that violate this policy.

8. *Test Case:* A test case denotes a concrete input that the fuzzer feeds to the PUT during a fuzzing campaign. The resulting execution is then judged by the bug oracle and the test case recorded in case the bug oracle deems it a violation of the security policy.

9. *Fuzz Configuration:* A fuzz configuration comprises the parameter values that control the fuzzing algorithm.

1.1.3 Taxonomy

Fuzzers can be roughly divided into three classes, defined by the amount of information from the execution of the PUT available to them. [24].

Black-box fuzzing

Black-box fuzzers have no access to the internal behavior of a PUT during execution. They can only make use of the bug oracle to decide whether a given input violates the security policy or not. The core advantage of this approach is the wide applicability since no instrumentation is required. This lack of potential instrumentation overhead also benefits execution throughput. The obvious disadvantage is the lack of information about PUT behavior. This makes it difficult for fuzzers to generate high-coverage inputs.

White-box fuzzing

In constrast to pure black-box fuzzers, white-box fuzzers have full access to the internal logic of the PUT as well as usually any original source code or documentation which can greatly aid the fuzzing process. In practice, this allows an approach known as *symbolic execution*: instead of relying on actual execution traces, a white-box fuzzer can gather

symbolic constraints at branching points and use constraint solving to find inputs that exercise a specific path. While this allows to easily find test cases of interest in theory, this approach scales poorly to large PUTs due to the path explosion problem. A popular white-box fuzzer is SAGE [**TODO cite godefroid12**].

Grey-box fuzzing

Since full information is often not available but observing certain parts of the internal behavior of the PUT might still be possible, *grey-box* fuzzing represents an important middle-ground between the previous two extremes. By making use of runtime instrumentation information like code coverage, the fuzzer can better judge the behavior of the PUT for a given input and direct its exploration efforts towards promising regions. Limitations are discussed in detail in Section 1.1.6.

Hybrid approaches

There also exists a variety of hybrid approaches combining different fuzzing methodologies. A prominent example is *Driller* that alternates pure greybox fuzzing with *concolic* execution (a combination of concrete and symbolic execution) to overcome hard branch conditions like CNC checks that have historically presented a major obstacle for fuzzers [46]

1.1.4 Input generation

The content of a test case directly controls whether a bug is triggered or not, making it one of the most influential design decisions of a fuzzer [26]. Input generation can be categorized into either generation- or mutation-based.

Generation-based input generation

Generation-based input generation makes use of a model of the input expected by the PUT, often represented through a grammar specification. These models can either be *predefined*, *inferred*, or *encoder-based*.

Predefined models are specified by the user. They are often used by network fuzzers consuming protocol specifications or kernel fuzzers consuming system call interfaces.

Inferred models don't rely on an explicitly specified model but rather infer it through some automatic means. Inference can either take place as a preprocessing step or during a fuzz campaign. Test-Miner [**TODO toffola17**] searches for literals inside of the PUT and derives suitable inputs. PULSAR[**TODO wressnegger15**] infers a network protocol from a set of captured network packages generated by the PUT.

Encoder-based models interpret the PUT as a decoder for inputs generated by an existing encoder.

Mutation-based input generation

Generating purely random inputs is not an effective strategy. In general, a PUT contains checks that reject invalid inputs, making it difficult to reach code paths that violate a given

security policy [26]. This motivates the use of a *seed corpus*, a set of well-structured inputs to the PUT like valid network packages or files. The seeds inside of the seed corpus are then successively mutated with the goal of triggering abnormal behavior. Mutation operators are divided into four sets:

1. *Bit-flipping:* Bit flipping is governed by a *mutation ratio*, the fraction of bits to flip per execution.
2. *Arithmetic mutation:* Arithmetic mutation reinterprets byte sequences as integers and perturbs them.
3. *Block-based mutation:* Block-based mutation inserts, deletes, replaces, permutes or resizes byte blocks, and splices blocks across seeds.
4. *Dictionary-based mutations:* Dictionary-based mutations substitute values with other values of high semantic weight. These usually include numbers like 0, or -1, or format strings for string substitution.

1.1.5 Coverage-Guided Greybox Fuzzing

This work will focus primarily on coverage-guided greybox fuzzing, *CFG* for short. We will outline the general approach following Böhme et al. [**TODO cite boehme17**].

Overview

A coverage-guided greybox fuzzer in general uses mutational input generation. For this, it keeps a set of test cases T as part of its runtime state. This set is initialized from a seed corpus S. The fuzzer then selects an input $t \in T$ and derives a new input t'. If the fuzzer deems the execution of the PUT under t' to be interesting, t' is then added to the test case set T.

The fuzzer must therefore solve mainly three problems: selecting an appropriate test case, applying appropriate mutations and deciding whether to retain the mutated input or not based on PUT behavior.

Test case selection

Test case mutation

Test case retention

AFL++

One famous example of a coverage-guided greybox fuzzer is AFL++ [10]. It is a mutation-based fuzzer that iteratively improves the seed corpus through a series of mutations and selections. Mutations include operations like bit flips, substitutions and splicing. Newly generated inputs are then evaluated and added to the seed corpus for further mutation and selection if they produce new coverage. The success of a greybox fuzzer is strongly dependent on a high-quality initial corpus and appropriate seed mutation and selection strategies.

- Frame CGF as the dominant modern design, popularized by AFL and refined by AFL++ [10] .
- Core feedback loop, generalizing the excerpt above (describe as algorithm/figure) [26, 10]:
  - pick a seed from the corpus (seed scheduling / energy assignment),
  - mutate it into a batch of test cases,
  - execute the instrumented target on each test case,
  - bug oracle check: crash/hang → report,
  - coverage check: new coverage → add test case to corpus,
  - repeat.
- Evolutionary interpretation: corpus = population, coverage novelty = fitness function, mutation/splicing = genetic operators [40, 26].
- Coverage signal design:
  - granularities: basic-block vs. edge/branch coverage, with bucketed hit counts [10],
  - AFL-style fixed-size bitmap indexed by hashed edge IDs: fast but suffers hash collisions that alias distinct edges ,
  - alternatives: hardware branch tracing (Intel PT) avoids compile-time instrumentation [56],
  - relevant later: MOGI's basic-block hit tracker uses the same hash-into-buckets idea (Chapter 5).
- Instrumentation options: compile-time insertion, static binary rewriting, or dynamic translation under emulation for closed-source binaries [10] .
- Bug oracles beyond crashes: sanitizers (e.g. AddressSanitizer) surface silent memory corruption, at significant runtime overhead [57] .
- Engineering for throughput: fork-server and persistent modes, snapshotting; scaling across cores is itself nontrivial (program-adaptive parallel fuzzing) [60].
- Scheduling refinements: power schedules biasing energy toward rarely exercised paths ; learned seed scheduling [51].
- Directed greybox fuzzing: optimize distance to chosen code sites instead of global coverage — e.g. for patch testing or reaching suspected-vulnerable code [6, 23] .

cite: original AFL (Zalewski)

cite: CollAFL (Gan et al. 2018) for the collision analysis and fix

cite: QEMU (Bellard 2005) for dynamic binary translation

cite: original AddressSanitizer paper (Serebryany et al. 2012)

cite: AFLFast (Böhme et al.

1.1.6 Strengths and Limitations of Coverage-Guided Fuzzing

- Strengths (why CGF won):
  - simple, robust, almost no per-target manual effort [26],

  - scales linearly with compute; embarrassingly parallel in principle [60],
  - empirically the most productive bug-finding technique of the last decade [26, 10].

- Limitations — each one motivates "smarter" input generation and frames a design goal of this thesis:
  - *Coverage plateaus*: random mutation practically never satisfies narrow constraints (magic bytes, checksums, length fields), so campaigns stall on frontier branches [46, 40].
  - *Coverage is only a proxy*: maximizing edges is not the same as triggering bugs; state- and data-dependent bugs need specific values in specific contexts, not just new edges [6].
  - *Blind mutation wastes the execution budget*: most mutations are irrelevant to control flow; which bytes and operators matter is input- and target-dependent [25, 3].
  - *Feedback aliasing*: bitmap hash collisions and coarse hit-count bucketing hide genuinely new behavior .
  - *Statelessness*: the classic loop assumes one self-contained input per fresh execution — breaks down for stateful/reactive targets (Section 1.1.7).
  - *Evaluation is tricky*: campaigns are highly stochastic; sound comparisons need many trials and statistical testing (also needed for Section 6.2).

cite: CollAFL

cite: Klees et al. 2018, "Evaluating Fuzz Testing"

- Research responses (survey briefly, as context for the RL approach):
  - data-flow/taint guidance to identify influential input bytes (GreyOne) [11],
  - neural surrogates: smooth the coverage function and follow gradients (NEUZZ, MTFuzz) [40, 41], deep-learning path prediction (NeuFuzz) [53],
  - hybrid concolic execution (Driller) [46],
  - machine learning across all loop stages — seed selection, mutation, scheduling — surveyed in [52, 34].

1.1.7 Stateful and Protocol Fuzzing

- Many high-value targets are reactive servers speaking a protocol (network services, industrial control systems, embedded devices): behavior depends on *session state* accumulated over a message exchange [1, 4].

- Consequences for the classic single-input model:
  - a test case is a *sequence* of messages, not one blob; ordering and protocol context matter,
  - deep states are reachable only through valid message prefixes; mutating a late message is useless if an early one breaks the session,

  - the same code can behave differently in different states, so pure edge coverage under-represents state coverage [4],
  - state is invisible: the fuzzer must infer or track it (response codes, annotated state variables) .
  - state-awareness matters in adjacent domains too, e.g. black-box web scanning [8].
- Extra decision problem on top of the CGF loop: *which state to fuzz next* — an exploration/exploitation trade-off, already modeled as a multi-armed bandit in prior work [4]; direct bridge to Section 1.2.
- Practical complications for encrypted/checksummed transports (e.g. QUIC): the fuzzer must sit inside the crypto boundary to mutate meaningfully [1].
- Position of this thesis: protocol-level actions against a stateful target are exactly the action space handed to the RL agent (Chapter 3).

cite: AFLNet (Pham et al. 2020) and SGFuzz (Ba et al. 2022) — the standard stateful greybox fuzzers; both are baselines the thesis should discuss

1.1.8 Emulator-Based Fuzzing

- Motivation from embedded/firmware targets: fuzzing on the device is slow and, worse, faults are often *silent* — without MMU protection or sanitizers, memory corruption frequently does not crash ("what you corrupt is not what you crash") [32].
- Rehosting the target in an emulator restores the missing capabilities [32] :
  - full observability: coverage and memory-access instrumentation without source access,
  - determinism and replayability of executions,
  - fine-grained fault detection via emulator-level checks,
  - *snapshots*: capture full machine state once, restore cheaply before every test.
- Snapshot restore gives an exact, cheap *reset* of the target to a known state — for stateful targets this is what makes per-episode experimentation tractable, and it is precisely the environment-reset primitive that reinforcement learning assumes (Section 1.2); realized here by the MOGI framework (Chapter 5).
- Cost: emulation overhead reduces raw executions per second versus native execution — sharpens the need to make every execution count (sample efficiency, Chapter 2).

cite: QEMU (Bellard 2005); Unicorn engine; optionally a rehosting survey (e.g. Wright et al. 2021 or Fasano et al. 2021)

1.2 Reinforcement Learning

1.2.1 Agent, Environment, and Markov Decision Processes

Reinforcement learning (RL) is an extremely general framework for approaching problems in which an agent strives to maximize a reward signal through a selection of actions in certain states [48]. These kinds of problems can be modeled as finite Markov decision processes (MDPs). A Markov decision process is a 3-tuple $MDP = (S, A, p)$ where

- $\mathcal{S}$ is the set of states,
- $\mathcal{A}$ is the set of actions where $\mathcal{A}_s \subseteq A$ are actions available in state $s \in \mathcal{S}$,
- $p(s', r|s, a) = \Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$ describes the system dynamics, i.e., the probability of encountering a next state s' and reward r at time step t when taking action a in state s at time step $t-1$

An RL agent's goal is to learn a policy $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ mapping each state s to probabilities of taking each possible action a to maximize the cumulative reward.

- Distinguish from supervised learning: no labeled correct actions, only evaluative (and possibly delayed) feedback; trial-and-error credit assignment [48]; idea traces back to early AI [28]; add [33] as the standard MDP reference next to the excerpt.
- Extend the excerpt's formalism [48]:
  - Markov property: transition depends only on current state and action,
  - episodic vs. continuing tasks; discount factor γ and discounted return $G_t = \sum_k \gamma^k r_{t+k}$,
  - state value V^π, action value Q^π; Bellman (optimality) equations; optimal policy π^*.
- Partial observability: agent often sees an *observation*, not the full state (POMDP setting) [48] — relevant later: a coverage vector is a very partial view of the emulated program's true state (Chapter 3).

1.2.2 Exploration versus Exploitation

- Fundamental dilemma: exploit the best-known action or explore to find better ones [48].
- Minimal setting: multi-armed bandit (single state); ε-greedy and optimism-in-the-face-of-uncertainty / UCB-style strategies [48, 22].
- Contextual bandit as intermediate step between bandit and full MDP — mirrors the ladder of evaluation targets in Chapter 4 (Sections 4.1 to 4.3).
- Foreshadow: fuzzing has exactly this structure — keep hammering a productive protocol state vs. probe untested ones [4].

1.2.3 Algorithm Families

Algorithms for finding an optimal π^* can be divided into three classes: value-based, policy-based and actor-critic.

Value-based methods try to learn a value function $Q : \mathcal{S} \to \mathcal{R}$ mapping states to their expected cumulative reward so that in each step, the action that maximizes the expected reward can be taken. These include well-known methods such as Q-learning, TD(λ) or SARSA.

Policy-based methods try to directly construct an optimal policy π^* without the use of a value function. They have various advantages. For example, they can easily handle continuous action spaces and ensure stochasticity which is important for state space exploration. A well-known classical policy-based algorithm is REINFORCE.

Actor-critic methods synthesize value- and policy-based methods by including information about a state's value in the policy learning process.

Since classical, tabular methods don't work well with large state spaces, state-of-the-art methods make use modern-day deep learning advancements. Examples include the family of Deep-Q Network (DQN) algorithms [29, 19] approximating the Q function by neural networks, policy-based gradient-descent methods like Proximal Policy Optimization (PPO) [37] and actor-critic methods like A3C [30]. Impressive results range from the master-level Go program AlphaGo [42] and its more generalized successors AlphaZero [43] and MuZero [36] to Dreamerv3 [16] that is able to learn how to acquire diamonds in the popular video game *Minecraft* without any human input or expert knowledge.

cite: Williams 1992, "Simple statistical gradient-following algorithms for connectionist reinforcement learning" (was williams1992 in the proposal)

- Foundational citations to attach to the excerpt's method names: temporal-difference learning [47]; Q-learning .
- Further DQN refinements between DQN and Rainbow: double DQN [17], dueling networks [54], prioritized experience replay [35] (PER returns in EfficientZero, Chapter 2).
- Further policy-gradient/actor-critic methods: TRPO [38], SAC [14].
- Broad algorithm surveys [12].
- Practical caveats of deep RL: high variance across seeds, hyperparameter sensitivity, reproducibility concerns [18] — informs the evaluation methodology in Section 6.2.

cite: Watkins & Dayan 1992, "Q-learning"

1.2.4 Model-Free versus Model-Based RL

- Model-free: learn policy/values directly from transitions; conceptually simple, but notoriously sample-hungry — millions to billions of environment steps [29, 31].
- Model-based: additionally learn a *dynamics model* of the environment and use it for planning or imagined training — the main lever for sample efficiency [31, 21].
- World-model line: learn compressed latent dynamics, train the policy inside the model [13, 15, 16].
- Planning-with-search line:
  - Monte-Carlo tree search (MCTS) [7], UCT: bandit-based selection inside the tree [22], modern review [49],
  - AlphaGo: MCTS + learned value/policy networks, with known rules [42]; AlphaGo Zero: self-play only [44]; AlphaZero: generalized across games [45],
  - MuZero: replaces the known simulator with a *learned latent model* — planning without knowing the rules [36],

  - EfficientZero: MuZero made sample-efficient (self-supervised consistency, value prefix, model-based off-policy correction); strong Atari performance from only 100k steps (≈ 2 hours) of experience [55] — treated in depth in Chapter 2.

1.2.5 Sample Efficiency

- Define: performance achieved per environment interaction; the binding constraint whenever a step is expensive [31, 55].
- In this thesis each environment step is a program execution under emulation (Section 1.1.8) — orders of magnitude slower than a simulator frame, so the fuzzing use case sits squarely in the low-sample regime that EfficientZero targets [55].
- This is the central argument for choosing a model-based, planning-heavy algorithm over the model-free methods used by most prior RL fuzzers (Section 1.3).

1.3 Reinforcement Learning for Fuzzing

1.3.1 Fuzzing as a Reinforcement Learning Problem

- Why the fit is natural: fuzzing is sequential decision making under uncertainty over an enormous search space, with a cheap scalar progress signal (new coverage) — exactly the RL interface of Section 1.2.1; first formalized as an MDP by Böttinger et al. [5].
- The generic correspondence (present as a table; concrete instantiation for this thesis in Chapter 3):
  - *environment* = the target program (plus harness/emulator),
  - *state/observation* = execution feedback: coverage information, program or protocol state, current input [5, 4],
  - *action* = a fuzzing decision: mutation operator/position, seed choice, protocol message [5, 3, 4],
  - *reward* = newly discovered coverage (or crashes, execution-time properties) [5, 50],
  - *episode* = one test case or one protocol session, ended by a target reset.

1.3.2 Prior Work

- Organize the survey by *which decision in the fuzzing loop* is learned (following the taxonomy of ML-for-fuzzing surveys [52, 34]):
- **Where/how to mutate**:
  - deep Q-learning over mutation actions on input substrings [5],
  - Rainfuzz: PPO-trained heat maps scoring byte positions by expected usefulness, run collaboratively with AFL++ [3],

  - (non-RL baseline for the same problem: MOpt's particle-swarm mutation-operator scheduling [25]).

- **Which seed/task to schedule**:
  - multi-armed-bandit / hierarchical seed scheduling for greybox fuzzing [51],
  - SyzVegas: MAB-based task and seed selection inside the Syzkaller kernel fuzzer [50],
  - Alphuzz: MCTS over the seed-mutation tree [58] — closest prior use of tree search, but over seeds, not program interaction.

- **Which grammar construct / input token to use**: BanditFuzz for SMT solvers [39]; BertRLFuzzer combines a language model with RL-chosen attack mutations [20].

- **Which protocol state to fuzz** (stateful setting, closest to this thesis): stochastic and adversarial MABs for state selection in industrial-protocol fuzzing [4].

- Adjacent security applications showing the same pattern: curiosity-driven web testing [59], Q-learning agents for SQL injection [9], DQN-based autonomous penetration testing [61].

1.3.3 Recurring Difficulties

- *Reward design*: coverage rewards are sparse and non-stationary — each edge pays out only once, so the reward distribution decays as the campaign progresses; motivates the adversarial-bandit view [4] and reward-model care in [50].

- *Observation design*: what the agent sees (raw bytes, coverage bitmap, protocol responses) is a heavily aliased, partial view of true program state (cf. Section 1.2.1) [5].

- *Throughput mismatch*: a classic fuzzer executes thousands of inputs per second; NN inference per decision must earn back its latency — one reason prior work often keeps the RL component advisory/parallel to a conventional fuzzer [3].

- *Sample cost*: model-free learners need more interactions than a fuzzing campaign can afford, especially under emulation (Section 1.2.5).

- *Mixed empirical results*: learned components do not consistently beat well-tuned static heuristics — e.g. AFLNet outperformed both bandit models in [4]; deep-RL evaluation pitfalls compound this [18].

- Honest framing for the thesis: these difficulties define the burden of proof for the approach evaluated in Chapter 6.

1.3.4 The Gap Addressed by This Thesis

- Common shape of prior work: a *model-free* learner (Q-learning, PPO, bandits) tunes *one heuristic knob* of an otherwise conventional random fuzzer [5, 3, 51, 50, 39].

- Unexplored combination — the subject of this thesis:

  - a *model-based, planning* algorithm (EfficientZero: learned latent dynamics model + MCTS [55, 36]) as the decision maker,
  - acting *directly* at the protocol level against a *stateful* target — the agent drives the session, rather than advising a random mutator (Chapter 3),
  - on targets rehosted in an emulator (MOGI, Chapter 5), whose snapshots provide the exact, cheap episode resets and basic-block coverage observations the MDP framing requires (Section 1.1.8),
  - rationale: the learned model lets planning happen in latent space, amortizing expensive real executions — directly attacking the sample-cost and throughput objections above (Section 1.2.5).

- Research questions to state explicitly here (evaluated in Chapter 6):
  - Can EfficientZero learn useful policies on fuzzing-shaped reward landscapes (sparse, decaying, heavily aliased observations)?
  - How does the approach scale from toy targets (Chapter 4) up to a real protocol implementation (open62541, Section 4.4)?
  - Where does planning with a learned model help or fail compared to classic greybox baselines (Sections 6.3 and 6.4)?

Chapter 2

EfficientZero

Chapter 3

Fuzzing with EfficientZero

Chapter 4

Targets

4.1 Bandit

4.2 Contextual Bandit

4.3 Sequence

4.4 open62541

Chapter 5

MOGI Setup

Chapter 6

Analysis

6.1 Challenges

6.2 Evaluation

6.3 Advantages

6.4 Disadvantages

Chapter 7

Conclusion

Chapter 8

Future Work

Bibliography

[1] Kian Kai Ang and Damith C. Ranasinghe. *QUIC-Fuzz: An Effective Greybox Fuzzer For The QUIC Protocol*. en. arXiv:2503.19402 [cs]. Mar. 2025. DOI: 10.48550/arXiv.2503.19402. URL: http://arxiv.org/abs/2503.19402 (visited on 09/24/2025).

[2] Abhishek Arya et al. *OSS-Fuzz*. License: Apache-2.0. Google LLC. URL: https://github.com/google/oss-fuzz (visited on 07/19/2026).

[3] Lorenzo Binosi et al. "Rainfuzz: Reinforcement-Learning Driven Heat-Maps for Boosting Coverage-Guided Fuzzing:" en. In: Lisbon, Portugal: SCITEPRESS - Science and Technology Publications, 2023, pp. 39–50. ISBN: 978-989-758-626-2. DOI: 10.5220/0011625300003411. URL: https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0011625300003411 (visited on 11/08/2025).

[4] Anne Borcherding et al. "The Bandit's States: Modeling State Selection for Stateful Network Fuzzing as Multi-armed Bandit Problem". en. In: *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. Delft, Netherlands: IEEE, July 2023, pp. 345–350. DOI: 10.1109/eurospw59978.2023.00043. URL: https://ieeexplore.ieee.org/document/10190654/ (visited on 07/11/2025).

[5] Konstantin Böttinger, Patrice Godefroid, and Rishabh Singh. *Deep Reinforcement Fuzzing*. en. arXiv:1801.04589 [cs]. Jan. 2018. DOI: 10.48550/arXiv.1801.04589. URL: http://arxiv.org/abs/1801.04589 (visited on 11/05/2025).

[6] Xu Chen et al. "Critical Variable State-Aware Directed Greybox Fuzzing". en. In: ().

[7] Rémi Coulom. "Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search". en. In: *Computers and Games*. Ed. by David Hutchison et al. Vol. 4630. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 72–83. ISBN: 978-3-540-75537-1 978-3-540-75538-8. DOI: 10.1007/978-3-540-75538-8_7. URL: http://link.springer.com/10.1007/978-3-540-75538-8_7 (visited on 11/14/2025).

[8] Adam Doupe et al. "Enemy of the State: A State-Aware Black-Box Web Vulnerability Scanner". en. In: (2012).

[9] Laszlo Erdodi, Åvald Åslaugson Sommervoll, and Fabio Massimo Zennaro. *Simulating SQL Injection Vulnerability Exploitation Using Q-Learning Reinforcement Learning Agents*. en. arXiv:2101.03118 [cs]. May 2021. DOI: 10.48550/arXiv.2101.03118. URL: http://arxiv.org/abs/2101.03118 (visited on 11/05/2025).

[10] Andrea Fioraldi et al. "AFL++: Combining Incremental Steps of Fuzzing Research". en. In: ().

[11] Shuitao Gan and Chao Zhang. “GreyOne: Data Flow Sensitive Fuzzing”. en. In: ().

[12] Majid Ghasemi, Amir Hossein Moosavi, and Dariush Ebrahimi. *A Comprehensive Survey of Reinforcement Learning: From Algorithms to Practical Challenges*. en. arXiv:2411.18892 [cs]. Feb. 2025. DOI: 10.48550/arXiv.2411.18892. URL: http://arxiv.org/abs/2411.18892 (visited on 10/25/2025).

[13] David Ha and Jürgen Schmidhuber. “World Models”. en. In: (Mar. 2018). arXiv:1803.10122 [cs]. DOI: 10.5281/zenodo.1207631. URL: http://arxiv.org/abs/1803.10122 (visited on 11/10/2025).

[14] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. en. arXiv:1801.01290 [cs]. Aug. 2018. DOI: 10.48550/arXiv.1801.01290. URL: http://arxiv.org/abs/1801.01290 (visited on 10/17/2025).

[15] Danijar Hafner et al. *Learning Latent Dynamics for Planning from Pixels*. en. arXiv:1811.04551 [cs]. June 2019. DOI: 10.48550/arXiv.1811.04551. URL: http://arxiv.org/abs/1811.04551 (visited on 11/08/2025).

[16] Danijar Hafner et al. *Mastering Diverse Domains through World Models*. en. arXiv:2301.04104 [cs]. Apr. 2024. DOI: 10.48550/arXiv.2301.04104. URL: http://arxiv.org/abs/2301.04104 (visited on 11/05/2025).

[17] Hado van Hasselt, Arthur Guez, and David Silver. *Deep Reinforcement Learning with Double Q-learning*. en. arXiv:1509.06461 [cs]. Dec. 2015. DOI: 10.48550/arXiv.1509.06461. URL: http://arxiv.org/abs/1509.06461 (visited on 10/09/2025).

[18] Peter Henderson et al. *Deep Reinforcement Learning that Matters*. en. arXiv:1709.06560 [cs]. Jan. 2019. DOI: 10.48550/arXiv.1709.06560. URL: http://arxiv.org/abs/1709.06560 (visited on 11/05/2025).

[19] Matteo Hessel et al. *Rainbow: Combining Improvements in Deep Reinforcement Learning*. en. arXiv:1710.02298 [cs]. Oct. 2017. DOI: 10.48550/arXiv.1710.02298. URL: http://arxiv.org/abs/1710.02298 (visited on 10/09/2025).

[20] Piyush Jha et al. “BertRLFuzzer: A BERT and Reinforcement Learning Based Fuzzer”. en. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.21 (Mar. 2024). arXiv:2305.12534 [cs], pp. 23521–23522. ISSN: 2374-3468, 2159-5399. DOI: 10.1609/aaai.v38i21.30455. URL: http://arxiv.org/abs/2305.12534 (visited on 01/02/2026).

[21] Lukasz Kaiser et al. *Model-Based Reinforcement Learning for Atari*. en. arXiv:1903.00374 [cs]. Apr. 2024. DOI: 10.48550/arXiv.1903.00374. URL: http://arxiv.org/abs/1903.00374 (visited on 11/10/2025).

[22] Levente Kocsis and Csaba Szepesvári. “Bandit Based Monte-Carlo Planning”. en. In: *Machine Learning: ECML 2006*. Ed. by David Hutchison et al. Vol. 4212. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 282–293. ISBN: 978-3-540-45375-8 978-3-540-46056-5. DOI: 10.1007/11871842_29. URL: http://link.springer.com/10.1007/11871842_29 (visited on 01/21/2026).

[23] Jaeseong Kwon et al. “Optimization-Directed Compiler Fuzzing for Continuous Translation Validation”. en. In: *Proceedings of the ACM on Programming Languages* 9.PLDI (June 2025), pp. 627–650. ISSN: 2475-1421. DOI: 10.1145/3729275. URL: https://dl.acm.org/doi/10.1145/3729275 (visited on 09/24/2025).

[24] Hongliang Liang et al. “Fuzzing: State of the Art”. en. In: *IEEE Transactions on Reliability* 67.3 (Sept. 2018), pp. 1199–1218. ISSN: 0018-9529, 1558-1721. DOI: 10.1109/TR.2018.2834476. URL: https://ieeexplore.ieee.org/document/8371326/ (visited on 09/24/2025).

[25] Chenyang Lyu. “MOpt: Optimized Mutation Scheduling for Fuzzers”. en. In: ().

[26] Valentin J. M. Manes et al. *The Art, Science, and Engineering of Fuzzing: A Survey*. en. arXiv:1812.00140 [cs.CR]. Apr. 2019. DOI: 10.48550/arXiv.1812.00140. URL: http://arxiv.org/abs/1812.00140 (visited on 07/19/2026).

[27] Barton P. Miller, Lars Fredriksen, and Bryan So. “An empirical study of the reliability of UNIX utilities”. en. In: *Communications of the ACM* 33.12 (Dec. 1990), pp. 32–44. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/96267.96279. URL: https://dl.acm.org/doi/10.1145/96267.96279 (visited on 07/19/2026).

[28] Marvin Minsky. “Steps toward Artificial Intelligence”. en. In: *Proceedings of the IRE* 49.1 (Jan. 1961), pp. 8–30. ISSN: 0096-8390. DOI: 10.1109/JRPROC.1961.287775. URL: http://ieeexplore.ieee.org/document/4066245/ (visited on 08/06/2025).

[29] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. en. In: *Nature* 518.7540 (Feb. 2015), pp. 529–533. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/nature14236. URL: https://www.nature.com/articles/nature14236 (visited on 09/13/2025).

[30] Volodymyr Mnih et al. *Asynchronous Methods for Deep Reinforcement Learning*. en. arXiv:1602.01783 [cs]. June 2016. DOI: 10.48550/arXiv.1602.01783. URL: http://arxiv.org/abs/1602.01783 (visited on 10/17/2025).

[31] Thomas M. Moerland et al. *Model-based Reinforcement Learning: A Survey*. en. arXiv:2006.16712 [cs]. Mar. 2022. DOI: 10.48550/arXiv.2006.16712. URL: http://arxiv.org/abs/2006.16712 (visited on 01/26/2026).

[32] Marius Muench et al. “What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices”. en. In: *Proceedings 2018 Network and Distributed System Security Symposium*. San Diego, CA: Internet Society, 2018. DOI: 10.14722/ndss.2018.23166. URL: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_01A-4_Muench_paper.pdf (visited on 07/11/2025).

[33] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. eng. Wiley Series in Probability and Statistics v.414. Hoboken: John Wiley & Sons, Inc, 2009. ISBN: 978-0-471-72782-8.

[34] Junyang Qiu et al. “A survey of coverage-guided greybox fuzzing with deep neural models”. en. In: *Information and Software Technology* 186 (Oct. 2025), p. 107797. ISSN: 09505849. DOI: 10.1016/j.infsof.2025.107797. URL: https://linkinghub.elsevier.com/retrieve/pii/S0950584925001363 (visited on 09/24/2025).

[35] Tom Schaul et al. *Prioritized Experience Replay*. en. arXiv:1511.05952 [cs]. Feb. 2016. DOI: 10.48550/arXiv.1511.05952. URL: http://arxiv.org/abs/1511.05952 (visited on 10/09/2025).

[36] Julian Schrittwieser et al. “Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model”. en. In: *Nature* 588.7839 (Dec. 2020). arXiv:1911.08265 [cs], pp. 604–609. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-03051-4. URL: http://arxiv.org/abs/1911.08265 (visited on 11/10/2025).

[37] John Schulman et al. *Proximal Policy Optimization Algorithms*. en. arXiv:1707.06347 [cs]. Aug. 2017. DOI: 10.48550/arXiv.1707.06347. URL: http://arxiv.org/abs/1707.06347 (visited on 10/17/2025).

[38] John Schulman et al. *Trust Region Policy Optimization*. en. arXiv:1502.05477 [cs]. Apr. 2017. DOI: 10.48550/arXiv.1502.05477. URL: http://arxiv.org/abs/1502.05477 (visited on 10/17/2025).

[39] Joseph Scott, Federico Mora, and Vijay Ganesh. “BanditFuzz: A Reinforcement-Learning Based Performance Fuzzer for SMT Solvers”. en. In: *Software Verification*. Ed. by Maria Christakis et al. Vol. 12549. Cham: Springer International Publishing, 2020, pp. 68–86. ISBN: 978-3-030-63617-3 978-3-030-63618-0. DOI: 10.1007/978-3-030-63618-0_5. URL: https://link.springer.com/10.1007/978-3-030-63618-0_5 (visited on 11/08/2025).

[40] Dongdong She et al. *NEUZZ: Efficient Fuzzing with Neural Program Smoothing*. en. arXiv:1807.05620 [cs]. July 2019. DOI: 10.48550/arXiv.1807.05620. URL: http://arxiv.org/abs/1807.05620 (visited on 11/05/2025).

[41] Dongdong She et al. “MTFuzz: Fuzzing with a Multi-Task Neural Network”. en. In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. arXiv:2005.12392 [cs]. Nov. 2020, pp. 737–749. DOI: 10.1145/3368089.3409723. URL: http://arxiv.org/abs/2005.12392 (visited on 01/17/2026).

[42] David Silver et al. “Mastering the game of Go with deep neural networks and tree search”. en. In: *Nature* 529.7587 (Jan. 2016), pp. 484–489. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/nature16961. URL: https://www.nature.com/articles/nature16961 (visited on 11/14/2025).

[43] David Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. en. arXiv:1712.01815 [cs]. Dec. 2017. DOI: 10.48550/arXiv.1712.01815. URL: http://arxiv.org/abs/1712.01815 (visited on 08/24/2025).

[44] David Silver et al. “Mastering the game of Go without human knowledge”. en. In: *Nature* 550.7676 (Oct. 2017), pp. 354–359. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/nature24270. URL: https://www.nature.com/articles/nature24270 (visited on 03/01/2026).

[45] David Silver et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. en. In: *Science* 362.6419 (Dec. 2018), pp. 1140–1144. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aar6404. URL: https://www.science.org/doi/10.1126/science.aar6404 (visited on 11/14/2025).

[46] Nick Stephens et al. “Driller: Augmenting Fuzzing Through Selective Symbolic Execution”. en. In: *Proceedings 2016 Network and Distributed System Security Symposium*. San Diego, CA: Internet Society, 2016. ISBN: 978-1-891562-41-9. DOI: 10.14722/ndss.2016.23368. URL: https://www.ndss-symposium.org/wp-content/uploads/2017/09/driller-augmenting-fuzzing-through-selective-symbolic-execution.pdf (visited on 11/05/2025).

[47] Richard S. Sutton. “Learning to predict by the methods of temporal differences”. en. In: *Machine Learning* 3.1 (Aug. 1988), pp. 9–44. ISSN: 0885-6125, 1573-0565. DOI: 10.1007/BF00115009. URL: http://link.springer.com/10.1007/BF00115009 (visited on 03/04/2026).

[48] Richard S. Sutton and Andrew Barto. *Reinforcement learning: an introduction*. en. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts London, England: The MIT Press, 2020. ISBN: 978-0-262-03924-6.

[49] Maciej Świechowski et al. “Monte Carlo Tree Search: A Review of Recent Modifications and Applications”. en. In: *Artificial Intelligence Review* 56.3 (Mar. 2023). arXiv:2103.04931 [cs], pp. 2497–2562. ISSN: 0269-2821, 1573-7462. DOI: 10.1007/s10462-022-10228-y. URL: http://arxiv.org/abs/2103.04931 (visited on 01/17/2026).

[50] Daimeng Wang et al. “SyzVegaS: Beating Kernel Fuzzing Odds with Reinforcement Learning”. en. In: (2021).

[51] Jinghan Wang, Chengyu Song, and Heng Yin. “Reinforcement Learning-based Hierarchical Seed Scheduling for Greybox Fuzzing”. en. In: *Proceedings 2021 Network and Distributed System Security Symposium*. Virtual: Internet Society, 2021. ISBN: 978-1-891562-66-2. DOI: 10.14722/ndss.2021.24486. URL: https://www.ndss-symposium.org/wp-content/uploads/ndss2021_6A-4_24486_paper.pdf (visited on 01/17/2026).

[52] Yan Wang et al. “A systematic review of fuzzing based on machine learning techniques”. en. In: *PLOS ONE* 15.8 (Aug. 2020). Ed. by Tao Song, e0237749. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0237749. URL: https://dx.plos.org/10.1371/journal.pone.0237749 (visited on 11/05/2025).

[53] Yunchao Wang et al. “NeuFuzz: Efficient Fuzzing With Deep Neural Network”. en. In: *IEEE Access* 7 (2019), pp. 36340–36352. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2903291. URL: https://ieeexplore.ieee.org/document/8672949/ (visited on 01/02/2026).

[54] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. en. arXiv:1511.06581 [cs]. Apr. 2016. DOI: 10.48550/arXiv.1511.06581. URL: http://arxiv.org/abs/1511.06581 (visited on 10/09/2025).

[55] Weirui Ye et al. *Mastering Atari Games with Limited Data*. en. arXiv:2111.00210 [cs]. Dec. 2021. DOI: 10.48550/arXiv.2111.00210. URL: http://arxiv.org/abs/2111.00210 (visited on 11/14/2025).

[56] Gen Zhang et al. "PTfuzz: Guided Fuzzing With Processor Trace Feedback". en. In: *IEEE Access* 6 (2018), pp. 37302–37313. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2018.2851237. URL: https://ieeexplore.ieee.org/document/8399803/ (visited on 01/02/2026).

[57] Yuchen Zhang et al. "Debloating Address Sanitizer". en. In: ().

[58] Yiru Zhao et al. "Alphuzz: Monte Carlo Search on Seed-Mutation Tree for Coverage-Guided Fuzzing". en. In: *Proceedings of the 38th Annual Computer Security Applications Conference*. Austin TX USA: ACM, Dec. 2022, pp. 534–547. ISBN: 978-1-4503-9759-9. DOI: 10.1145/3564625.3564660. URL: https://dl.acm.org/doi/10.1145/3564625.3564660 (visited on 01/17/2026).

[59] Yan Zheng et al. *Automatic Web Testing using Curiosity-Driven Reinforcement Learning*. en. arXiv:2103.06018 [cs]. Mar. 2021. DOI: 10.48550/arXiv.2103.06018. URL: http://arxiv.org/abs/2103.06018 (visited on 11/05/2025).

[60] Anshunkang Zhou, Heqing Huang, and Charles Zhang. "KRAKEN: Program-Adaptive Parallel Fuzzing". en. In: *Proceedings of the ACM on Software Engineering* 2.ISSTA (June 2025), pp. 274–296. ISSN: 2994-970X. DOI: 10.1145/3728882. URL: https://dl.acm.org/doi/10.1145/3728882 (visited on 09/24/2025).

[61] Shicheng Zhou et al. "Autonomous Penetration Testing Based on Improved Deep Q-Network". en. In: *Applied Sciences* 11.19 (Sept. 2021), p. 8823. ISSN: 2076-3417. DOI: 10.3390/app11198823. URL: https://www.mdpi.com/2076-3417/11/19/8823 (visited on 11/05/2025).